

POLYMARKET EVENT PRICE-FEED BOT

Implementation contract and chronological delivery plan

Eros Markets / EventPerp - CP-PRICE

Sports, politics and crypto event outcomes

Version 1.0 | 2 October 2026 | DESIGN FOR REVIEW

Target deliverable: one configurable offchain service that reads independently priced Polymarket prediction-outcome books, validates the approved event mapping and source data, calculates depth-N summaries, signs the risk engine's exact observation envelope and reliably delivers it to the correct isolated market engine.

Not delivered by this document: bot implementation, approved production calibration, deployed contracts, operational signing keys, live engine integration, human gate acceptance or permission to deploy.

Repository baseline: integration/risk at 250623536768101511999fd2223ba4ce38a6bdd3. Risk specification v1.1; economic baseline v1.0. Existing untracked contracts/foundry.lock was preserved.

User-selected source: Polymarket prediction-outcome CLOB data. No source substitution, category-based probability model or own-perp index is authorized.

This plan follows the existing documents' task/dependency/gate/evidence style. Requirement labels distinguish fixed repository behavior from proposed bot engineering and unresolved integration decisions. All implementation tasks start NOT STARTED.

Important boundary: CLAUDE.md excludes price-feed internals from the current risk-team coding scope. This artifact proposes a separate CP-PRICE workstream. A human must record its owner and permitted implementation location before a coding agent starts that work. No shared rule, approval fingerprint, gate record or contract was changed to create this plan.

Reading map and status

How to use this plan

Read pages 3-7 to agree scope, event eligibility and configuration; pages 8-15 define collection, arithmetic, the signed output and delivery; pages 16-20 cover failure handling, tests, calibration and milestones. Pages 21-27 are the executable backlog. Pages 28-31 contain decisions, completion criteria, sources and the plan self-review.

Pages	Subject	Intended reader
3-4	Authority, boundaries and architecture	CP-PRICE owner and risk reviewer
5-7	Admission, configuration and categories	Listing, oracle and feed owners
8-10	Data acquisition, freshness and exact math	Collector developer
11-12	Observation ABI and raw signing	Risk and signer developers
13-15	Persistence, relay and operations	Feed developer and operator
16-18	Failure states, vectors and integration	Reviewer and test owner
19-20	Calibration and chronological gates	Risk, production and integrator
21-27	PF001-PF028 atomic tasks	Implementer
28-31	Open decisions, done criteria, sources and self-review	All approving owners

Status vocabulary

FIXED [R]: required by the current repository spec/interface or executable code. **PROPOSED [P]:** a bot design choice in this document, to be reviewed and pinned before implementation. **OPEN [Q]:** not settled by the reviewed source; dependent production work must stop until a named owner decides it. **OBSERVED [O]:** limited read-only evidence from earlier API checks, not a production guarantee.

PF-G0 through PF-G7 are proposed price-feed milestones, not the repository's G0-G7. They do not refresh A043 fingerprints, accept risk G7, approve a main merge or authorize deployment.

Starting position

The risk-side ingress, observation store and guards exist. CP-PRICE collector/signing agreement remains blocked in repository STATUS. Internal risk reviews and human risk-G7 acceptance are separate pending work. CLOB/oracle internals belong to their own teams. Older HANDOFF/release artifacts contain historical test counts and status text; current STATUS and source code take precedence when those differ. [R01, R02, R09, R10]

The plan can be reviewed now. After scope authorization, deterministic fixtures, collection and dry-run output can be built without completed live CLOB/oracle components. Complete economic and production integration cannot be declared until the real counterparts and approved environment exist.

01. Authority, scope and non-negotiable rules

Source hierarchy

Follow CLAUDE.md, risk_spec.md, the gate/status documents, counterpart interfaces and the existing implementation plan. Historical PDFs and old lane restrictions are not new economics. Do not reopen DEC-01 through DEC-14. CP-PRICE integration must confirm assumption I-3 instead of inventing a replacement signature protocol. [R01-R03, R09]

What the bot owns

Approved external-market mapping; public read adapters; source-data validation; integer normalization and approved impact-price calculation; raw snapshot evidence; durable observation sequencing; raw-digest signing; transaction relay; receipt/reorg recovery; health monitoring and operator runbooks. It is a production-intended component, developed first in deterministic and read-only modes. [P]

What stays inside risk

Authentication, ordering, depth/spread checks, index TWAP, own-perp observations, basis, normal mark, freshness guards, funding stops, bootstrap admission, liquidation and INVALID capture. The bot does not send a precomputed 300-second index or final mark in place of a depth-N sample. It never writes balances, positions, final outcomes, market configuration or risk parameters. [R04-R07]

What belongs to other owners

Oracle: event evidence, authoritative halt/finality and outcome decisions. Factory/listing: engine identity, pinned source/signer/rules and immutable N/spread/T/INVALID configuration. Risk/calibration: economic parameter approval and margin/leverage evidence. Production: chain facts, gas, keys, observability ownership, audit and release decision. Human reviewers: required acceptance records and main merge. [R01-R03, R08-R10]

Required behavior

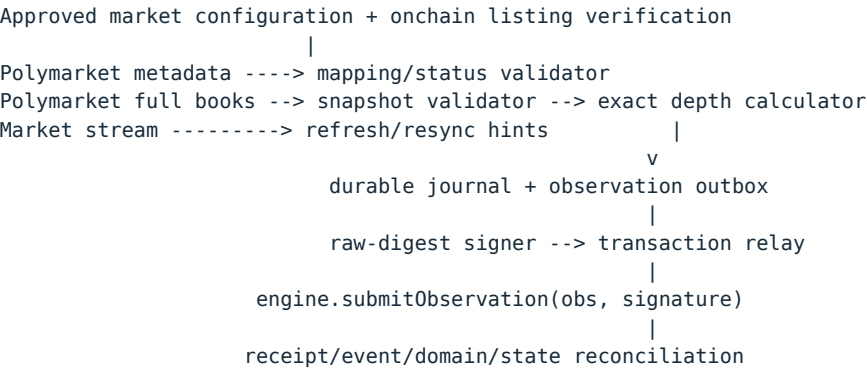
- Independent reference source; never this engine's perpetual book.
- Times in Unix seconds; prices in WAD; quantities in lots. No floating-point financial math.
- Authentic source time is preserved; receiving or submitting old data does not refresh it.
- Every market has explicitly approved semantics and independent persistent state.
- Failure is visible unavailability, not a guessed price, last-trade fallback or silently substituted venue.
- A mock/local test is not a live counterpart pass. No deployment of any kind is authorized.

Scope stop: obtain written CP-PRICE ownership/path authorization before creating bot code. Proposed location: a dedicated price-feed package or repository chosen by the team. This plan does not grant access to counterpart internals or modify CLAUDE.md. [Q01]

02. Unified service architecture

One service; isolated market workers

Use a modular service with shared provider code, not separate cricket/football/politics binaries. Each worker is keyed by chainId + engineAddress + marketId + sourceId and owns its own source history, observation sequence, lifecycle and outbox. A shared transaction account also needs a globally serialized nonce manager. One market's outage must not block all workers. [P]



Health, raw evidence and alerts observe every stage.

Module responsibilities

Module	Responsibility
config / admission	Schema, rules approval, engine-listing equality
adapters / polymarket	Metadata and complete outcome-book reads
validate / time	Identity, decimals, source clock and integrity
math / impact	Approved exact depth-N summaries and midpoint
journal / outbox	Atomic evidence, sequence allocation and retries
signer	Allowlisted raw digest; no arbitrary signing endpoint
relay / reconcile	Nonces, RPC, mined events and reorg handling
observe / cli	Read-only inspection, health and operator commands

Recommended starting stack [P]: TypeScript with bigint financial math, a pinned supported Node runtime, a reviewed EVM encoding/signing library and a transactional SQL journal. Start single-active-writer; choose SQLite for a measured single-node pilot or PostgreSQL for multi-instance coordination. Freeze versions/storage choice at PF-G0; no library version or infrastructure capacity is silently approved here. Python Fraction is the independent test reference, not the production ledger.

No new standalone feed contract or cross-chain token bridge is required by the documented interface: the service reads offchain data and delivers signed observations to the existing engine. [R04-R06]

03. Market admission before activation

Admit an exact payoff, not a category

An operator may discover candidate markets by sports/politics/crypto tags. Discovery never auto-activates a worker. Admission requires an approved dossier with the Eros question, external question/condition/outcome-token identities, exact YES mapping, resolution source, deadline and exception comparison. Title similarity, matching team names, popularity and volume are insufficient. [P; R02, R08]

Admission step	Required evidence
Identity	Correct external event, condition, token and outcome label
Payoff	Approved correspondence of the Eros YES claim to the token
Clocks	UTC deadline and source rules; metadata date is not authority
Exceptions	Draw, cancellation, postponement, void and ambiguity comparison
Liquidity	Both sides can support candidate N with acceptable spread
Freshness	Authentic samples and delivery support required coverage
Lifecycle	Source availability through the required observation period
Permissions	Data-access/usage terms reviewed; no restriction bypass
Listing	Signer, source/rules hashes, N/spread and engine address verified

Fixed listing limits

The engine requires $T \geq \text{listedAt} + 86,400 \text{ seconds}$, and $T + \text{captureGraceSecs} \leq \text{listedAt} + \text{voidSecs}$. With the new-listing production profile's 3,600-second grace and 30-day void schedule, the horizon is 24 hours through 29 days 23 hours. Newly listed five-minute/hourly crypto contracts and same-day matches with less than 24 hours remaining are not eligible under this baseline. Do not move T or lengthen the profile silently. [R02, R08]

Polymarket's stated endDate may differ from the actual event semantics. A presidential-election contract can include a runoff; a crypto contract can distinguish touching a threshold from closing above it. Record those clauses explicitly.

Reject or quarantine

Unmapped/multi-leg outcomes, uncertain deadlines, incompatible exception payouts, rules clarifications, unapproved negative-risk semantics, missing source timestamps, unavailable executable depth and an engine/listing mismatch all block activation. Preserve the dossier and reason code. A risk/oracle owner must explicitly review unresolved semantic differences; the feed developer cannot rewrite either market's rules. [Q06]

04. Configuration contract

Proposed offchain configuration schema [P]

All financial integers, token IDs, sequence numbers and Unix times serialize as validated decimal strings. Bytes32 fields require exactly 32 bytes and addresses require valid EVM addresses. Missing required values block activation. Production values come from approved manifests, never test defaults.

Group	Required fields
Identity	schemaVersion, configVersion, category, displayLabel, enabled
Destination	chainId, engineAddress, engineCodeHash, ABI hash/version, marketId
Source pinning	sourceId, sourceRulesHash, expectedSignerAddress
Polymarket mapping	eventId, marketId, conditionId, outcomeTokenId, outcomeLabel
Semantics	erosRulesHash, externalRulesDigest, mappingApprovalId, scheduledT
Pricing	depthNLots, maxSpreadWad, impactMethodId, roundingPolicyId
Units	quoteNormalizationId, sourceQuantityUnit, lotConversionPolicyId
Freshness	timestampPolicyId, polling/relay budget profile, metadata-age bound
Lifecycle	requiredFeedUntil, listed INVALID rule identity, closure policy
Operations	RPC/source endpoint allowlists, signer key reference, relay profile

Category is one of sports, politics or crypto. Optional subcategory is cricket, football, elections, BTC-event, etc. These labels support discovery and operations; they do not change the observation ABI or choose an oracle/risk template automatically.

Startup checks

Read listing() and sourceState(sourceId) at a labeled block. Require equality for marketId, indexSourceId, indexSigner, indexRulesHash, depthNLots, maxSpreadWad, scheduledT and disclosed INVALID settings; verify chain, deployed code and ABI identity. Compare the approved mapping and actual external metadata before any signing. Fail closed if the configured source is not the engine's index source. [R04-R08]

Hash and change policy [Q04]

Agree a canonical source-rules document and hashing format before initialization. It should bind venue, exact outcome mapping, time policy, units, impact method, rounding and failure semantics; determine which values belong in sourceHash, rulesHash and indexRulesHash with factory/risk owners. Do not assume a Polymarket condition ID equals any Eros hash.

Immutable engine pins cannot be changed by a bot config edit. A source/signer/rule/N change requires the owners' approved new-listing/version process. Operational timeout changes remain bounded by the approved economic policy and are audited.

05. Sports, politics and crypto profiles

Shared collection pipeline

All three categories use the approved outcome token's prediction CLOB book. The category-specific work is mapping and rule review, not writing a different price calculator. Polymarket's discovery API can identify events and outcomes; approval remains explicit. [W03, W04]

Category	Illustrative YES claim	Mandatory semantic checks
Cricket	India wins a specified match	Match/competition, tie, no-result, abandonment, DLS, reserve day, postponement
Football	Arsenal wins a specified match	Fixture, regulation vs extra time/penalties, draw outcome, cancellation, reschedule
Politics	Candidate wins a specified election	Office/jurisdiction, runoff, certification vs reporting, withdrawal, recount, deadline
Crypto	BTC above X at timestamp T	Above vs at/above; hit-before vs close-at; quote pair, source venue, candle, UTC conversion

These are schematic examples, not approved markets or live configuration files. Every actual token ID, deadline and rule digest must be resolved from evidence and reviewed.

Correct price input

For a BTC-above-threshold event, read the YES event-claim book with price in [0,1], not BTC/USD at tens of thousands of dollars. Underlying exchange prices may be evidence for the resolution oracle; they are not this bot's probability input. Likewise, a live football score or polling percentage cannot replace the event outcome's executable book.

Outcome labels

Sports outcomes may be team names rather than literal Yes/No. Record the approved token-to-Eros-YES mapping explicitly; never select the first outcome array element without comparing labels/identities. Do not invert a NO book with 1-price unless that complement mapping, bid/ask transformation and semantics are explicitly reviewed. Prefer the directly mapped YES-equivalent token.

Exception and closure policy

Some sports contracts settle 50/50 on cancellation, whereas Eros INVALID uses its listed 24-hour index rule or disclosed fallback. Treat this as a semantic review question, not as proof of equivalence or permission to change DEC-08. Polymarket resolution and rule clarifications must be monitored, but they do not authorize this bot to call settle(), halt() or choose an INVALID price. [R02, R03; W05]

For an outcome book that closes before the required Eros recording period ends, record the resulting gap and execute the approved missing-data procedure. Do not fabricate fresh 0, 1 or 0.5 observations from a final result.

06. Polymarket adapter and acquisition

Public read contract

Use Gamma for event/market metadata and the prediction CLOB for the approved token's complete book. Relevant response ingredients are condition/token identity, timestamp, bid/ask price-size levels, minimum order size, tick size and book hash. Prices and quantities arrive as strings; parse them exactly. The documented REST route is GET /book?token_id=... at clob.polymarket.com. [W01]

Discovery may use tagged event listings; deployed API versions/routes must be validated and pinned by contract fixtures before implementation. Follow current official schemas, including pagination. Do not assume an older SDK or endpoint exists merely because a prior example used it. [W03]

Initial acquisition design [P]

Make complete REST snapshots authoritative for the first implementation. Use the market stream to request prompt refreshes and detect lifecycle changes. Periodic snapshots continue even when prices do not change, subject to verified timestamp meaning and rate limits. No new timestamp is invented by the collector.

The documented market stream accepts token subscriptions at wss://ws-subscriptions-clob.polymarket.com/ws/market. It uses assets_ids and type=market, with application PING every ten seconds. That heartbeat demonstrates transport liveness, not a fresh executable book. [W02]

Do not synthesize authoritative depth from last-trade or best-bid/ask messages. If later implementing a stream-maintained complete book, first prove snapshot/delta ordering, deletion rules, reconnect boundaries and integrity. Quarantine on gaps or uncertainty; acquire a complete snapshot before resuming. No unverified delta-merging algorithm is part of v1.

Validation before calculation

- Correct condition and asset ID; expected outcome and nonnegative decimal sizes.
- Required timestamp and all complete-book fields; sane payload/body/level limits.
- Parse prices within [0,1], check tick grid, aggregate duplicate levels and remove zero-size levels under a tested policy.
- Sort bids descending and asks ascending using integer prices; do not rely on vendor array position.
- Check current trading status and approved metadata-age bound. Reject stale approval/rule identities.
- A vendor hash is recorded evidence, not proof of source honesty or a substitute for the Eros signature digest.

Retry boundedly with backoff/jitter and a shared provider limiter. HTTP 429, timeout, invalid JSON and schema drift are visible errors; queues are bounded so overload cannot silently turn old data into fresh samples.

07. Time, freshness and cadence

Fixed clock requirements [R]

$\text{observedAt} \leq \text{publishedAt} \leq \text{chain acceptance block.timestamp}$. observedAt is nondecreasing; sequence strictly increases. Future tolerance is zero. The sample carries over $[\text{observedAt}, \min(\text{nextObservedAt}, \text{observedAt} + 30))$. A complete 300-second index window must contain 300 covered seconds. Millisecond source times convert to seconds using integer floor after a reviewed timestamp policy. [R04-R07]

Open source-time policy [Q03]: confirm whether a Polymarket timestamp marks snapshot generation, last book mutation or another time; distinguish cache age from receipt time. The safest baseline preserves authenticated source time. If another definition of observedAt is proposed, risk must approve its evidence/freshness semantics and rules hash first. No local retimestamping of cached data.

Budget, not a guessed production interval

Let A = source age at receipt, C = calculation/signing delay, R = relay/inclusion delay and M = safety margin. Required live headroom: $A + C + R + M < 30$ seconds, measured against reliable clocks. Also keep successive approved observedAt gaps ≤ 30 seconds to avoid interval gaps. Chain timestamps and future checks require additional measured clock-skew handling.

A ten-second local fixture interval is a useful test seed, not an approved production cadence. Choose polling/signing interval, expiry/headroom threshold and RPC budgets from soak measurements. Use integer milliseconds for timing metrics. Fetch/send more often does not fix a timestamp that only advances on rare source mutations. [Q07]

Repeat and delayed data

An unchanged price can be a fresh observation only with a genuinely new approved observation time/evidence. An unchanged old timestamp must not create additional freshness. Within a second, the engine replaces an earlier sample; the bot should coalesce unsent candidates but never alter a signed payload. Preserve strict sequence and time ordering.

Delayed observations can authenticate successfully while offering no current freshness. They must not be displayed as live. Historical evidence may be archived; any intentional historical delivery requires a reviewed ordering/revision policy and cannot erase a funding stop. Recovery needs a complete valid window, not one tick. [R04-R07]

Continue independent index recording after an early economic halt when needed through T for INVALID; this does not resume trading or funding. Source closure/gaps invoke the listed policy, not a invented settlement price. [R02, R03]

08. Exact normalization and impact pricing

Required numeric boundary

WAD = 10^{18} ; 1 lot = 0.001 claim. Parse decimal strings as integers/rationals, never JavaScript Number or floating-point JSON prices. A price of 0.61 converts to 610000000000000000 WAD; five claims convert to 5,000 lots. Preserve source fractional quantities exactly during aggregation and floor only at the approved depth-summary boundary. Reject malformed signs, exponent strings or excessive precision unless the parser explicitly supports them. [R01, R02, R04]

Impact method is an integration decision [Q02]

The engine verifies signed summaries and midpoint; it does not walk the external book. The reviewed CP-PRICE documents do not freeze whether impact bid/ask means quantity-weighted execution price or the marginal price at N. Agree that method and conservative rounding with risk before finalizing the calculator/source hash.

Proposed VWAP branch, only if approved: sell N against bids descending; buy N against asks ascending. Fill each level up to remaining exact quantity. $\text{bidImpact} = \text{floor}(\text{sum}(\text{priceWad} * \text{takenClaims}) / \text{NClaims})$; $\text{askImpact} = \text{ceil}(\text{sum}(\text{priceWad} * \text{takenClaims}) / \text{NClaims})$. Compute both with exact rational/full-precision products. Floor available claim quantity * 1,000 to report available depth lots. Round bid down and ask up; never overstate liquidity. [P]

Alternative marginal branch, only if approved: select each side's level that completes N with the same quantity walk; use that level's normalized price. The illustrative multi-level vector on page 17 deliberately shows why this choice changes output.

Engine depth validity - reproduce exactly

$N > 0$; $\text{bidDepthLots} \geq N$; $\text{askDepthLots} \geq N$; $0 < \text{impactBidWad} \leq \text{impactAskWad} < 10^{18}$; $\text{impactAskWad} - \text{impactBidWad} \leq \text{maxSpreadWad}$. For a valid summary, $\text{priceWad} = \text{floor}((\text{impactBidWad} + \text{impactAskWad}) / 2)$. Equality at N/spread passes; endpoint/crossed/thin summaries are invalid. [R06]

Spread is an absolute claim-price difference, not a percentage of midpoint. Fixture 5e16 means 0.05 per claim. N=500 lots and spread=5e16 are local tests, not production recommendations. The same engine depth rule is used for independent index and internal perp observations, so calibrate with both owners. [R05, R08]

Minimum order size, fee treatment, size precision and quote denomination must be reflected in the approved executable-depth policy. Polymarket documents USDC-backed pUSD collateral; Eros uses USDC per claim. Agree normalization rather than assuming all stablecoin labels are interchangeable. [Q05; W06]

09. Exact output the risk engine accepts

External call

submitObservation(Observation obs, bytes signature)

The tuple contains exactly the following fields in this order. [R04]

Field	Solidity type	Bot responsibility
marketId	bytes32	Exact Eros market ID, not an external slug
sourceId	bytes32	Configured index source identity
sequence	uint64	Durable strictly increasing source sequence
observedAt	uint64	Approved genuine observation time, seconds
publishedAt	uint64	Actual finalized publish/signing time, seconds
priceWad	uint256	Floored midpoint of valid impact bid/ask
impactBidWad	uint256	Approved depth-N sell-side summary
impactAskWad	uint256	Approved depth-N buy-side summary
bidDepthLots	uint256	Conservatively normalized available bid depth
askDepthLots	uint256	Conservatively normalized available ask depth
sourceRulesHash	bytes32	Exact engine-pinned index rules hash

acceptedAt, depthValid and payloadDigest are NOT fields of the input tuple. The engine produces them in ObservationAccepted. chainId and engine address are signature-domain inputs, not additional Observation fields. Category, raw books, event labels and Polymarket hash stay in the offchain evidence/configuration. [R04, R05]

Accepted event

ObservationAccepted(sourceId indexed, sequence, observedAt, publishedAt, acceptedAt, priceWad, depthValid, payloadDigest). Match source/sequence and computed digest to the mined log; for valid input require the emitted midpoint to match the packet. Do not treat transaction success alone as valid depth or a ready index.

Invalid versus unauthenticated

A fresh authentic complete book with thin depth, crossed sides or excessive spread can produce a correctly signed invalid summary under the agreed invalid-packet policy. The engine records depthValid=false and midpoint zero, contributing no coverage. Proposed policy sends invalid summaries promptly to end prior carry, preserving actual fields and using priceWad=0 when invalid; risk must approve exact representation. [Q08]

Missing/untrusted timestamps, identity mismatches or incomplete books are not permission to fabricate an invalid observation time. Stop valid publication and alert; determine any explicit invalidation mechanism with risk. The engine's prior valid sample may carry for at most its remaining 30 seconds. [R05-R07]

10. Signing protocol and domain

Frozen raw digest [R05; R09 I-3]

```
TYPE_STRING =  
Observation(bytes32 marketId,bytes32 sourceId,uint64 sequence,  
uint64 observedAt,uint64 publishedAt,uint256 priceWad,  
uint256 impactBidWad,uint256 impactAskWad,uint256 bidDepthLots,  
uint256 askDepthLots,bytes32 sourceRulesHash,uint256 chainId,  
address engine)  
  
TYPEHASH = keccak256(UTF8(TYPE_STRING with NO line breaks))  
digest = keccak256(abi.encode(  
    TYPEHASH, marketId, sourceId, sequence, observedAt, publishedAt,  
    priceWad, impactBidWad, impactAskWad, bidDepthLots, askDepthLots,  
    sourceRulesHash, chainId, engineAddress  
))  
signature = raw secp256k1 signature over digest
```

The displayed type string wraps for readability; actual hashing uses the single exact string from `PriceIngress.sol`. Use `abi.encode`, NOT `abi.encodePacked`, and Keccak-256, NOT NIST SHA3-256. There is no EIP-191 personal-message prefix and no EIP-712 typed-data wrapper. A generic `signMessage/signTypedData` call is wrong unless it demonstrably signs this exact raw digest.

Signer contract [P]

Use a reviewed raw-digest-capable signer. Verify recovery locally against `expectedSignerAddress`; test `r/s/v` serialization against the actual Solady recovery path. A 65-byte `r || s || v` format matches B016's fixture; pin recovery-byte and low-s policy through differential tests. Never silently switch signature encoding to an untested compact format.

The ingress pins one signer per source at initialization and exposes no signer rotation path. A key-loss/compromise plan must therefore be reviewed before listing; do not promise hot key rotation. Transaction sender can be a separate relay key because authentication is based on signature recovery. [R05]

Required cross-check

For independently constructed vectors, compare local digest to `engine.observationDigest(obs)`; recover the expected address; submit through `submitObservation` in the approved local test environment. Mutate every field, chain and engine to prove rejection. Include wrong-prefix and wrong-encoding negatives. Check widths before serialization and halt before `uint64` sequence exhaustion.

Keys are never printed in logs, sample configuration or plan artifacts. Signer authorization binds approved domains/mappings and disallows arbitrary requests; archives store payload/digest/signature and key reference, not private key material.

11. Durable state and exactly-once effects

Proposed journal records [P]

Record	Contents / invariant
MarketConfig	Approved version, hashes, domain, rule dossier and lifecycle
RawSnapshot	Exact body, source time/hash, receipt time, HTTP metadata, identity
NormalizedBook	Decimal policy, canonical levels, raw-evidence digest
Observation	Eleven fields, method/config IDs, digest, signature, unique sequence
Outbox	Durable status, attempt history, relay nonce and transaction hashes
AcceptedLog	Block number/hash, transaction/log index, depth flag and digest
WorkerState	Next sequence, last source time, last accepted state, writer lease

Primary uniqueness: (chainId, engineAddress, sourceId, sequence); include marketId/config identity for auditing. A transaction nonce is NOT an observation sequence. Never share one blind sequence across isolated engines. Storage integrity and restore tests are part of feed correctness.

Atomic publication

Acquire single-writer ownership; validate snapshot/mapping; allocate next never-used sequence and persist immutable unsigned packet in one database transaction. Finalize publishedAt according to the approved publishing definition, sign, persist packet/signature before broadcasting, then enqueue relay. A crash at any step must not produce a second different packet under the same sequence. Discarding a sequence is allowed by the engine's strict greater-than rule; sequence reuse is not. [R05; P]

Publish-state contract must handle signing delay: if headroom expires before signing/broadcast, mark the packet expired and obtain a genuinely new snapshot/sequence. Do not edit an already signed publishedAt to disguise age. [Q03, Q07]

Recovery and concurrency

Before resuming, reconcile sourceState at a canonical block with journal packets and accepted logs. nextSequence must exceed all previously allocated/signed sequences, not merely the latest mined value. A restored backup behind the signing journal requires recovery, not guesswork. Unknown higher onchain sequences, unexpected signers or source times quarantine the worker.

Use a database-enforced lease/fencing mechanism or a single active process; the signer also enforces unique sequence/payload identity. Prevent two replicas from signing diverging packets. A process heartbeat without exclusive-writer enforcement is insufficient.

Archive invalid packets and source gaps as well as valid data. Retain enough raw evidence to reconstruct all tested live and INVALID windows and the listing lifetime under an approved retention policy. Do not assume the engine's live ring is a complete offchain audit archive.

12. Relay, receipts and chain recovery

Ordered delivery [P]

One delivery stream per engine/source; a shared relay account uses one serialized nonce allocator across every market. Simulate the exact call, estimate bounded gas and verify current chain/domain/listing before sending. Check observation age/headroom and future-clock ordering again. Use only approved RPC endpoints and spend limits. No broadcasting is authorized while producing this plan.

Submitting a newer observation before an older one makes the older sequence unusable. Do not bypass an unresolved in-flight packet without reconciling its receipt and canonical state. Coalesce only unsent candidates, preserving evidence; if dropping a known invalid transition would extend prior carry incorrectly, retain it or use an approved equivalent invalidation policy. [Q08]

Retry semantics

Network timeout is an unknown result, not a failed transaction. Search transaction/receipt, logs and sourceState. Re-send the identical signed observation if it is still admissible; gas replacement changes transaction fee fields, not the packet. A duplicate/old-sequence revert after a previous success is a reconciliation outcome, not a reason to allocate an arbitrary replacement price.

Keep bounded retry and age budgets. Expired candidates are archived rather than made fresh. If the chain is too slow for coverage, declare degraded/unavailable and alert; do not alter the freshness constant. Separate pending, mined and finalized observations.

Receipt criteria

Successful receipt plus the matching ObservationAccepted log and expected digest establishes mined delivery. depthValid=true establishes only a valid sample; it does not imply indexOk or markOk. Read indexTwap300/riskContext at a labeled block for readiness. Invalid packets may succeed with depthValid=false. [R04-R07]

Reorg policy [Q09]

Pin the actual chain's confirmation/finality policy with production. Record block hashes and invalidate orphaned acknowledgments. Re-read sourceState on the canonical chain; keep sequence reservations from all signed packets. Retry the original orphaned packet only if ordering/time policy still permits it, otherwise expire it and obtain new evidence. Do not assert that finality depth is known from this plan.

An RPC disagreement, unexpected code/ABI change, unexplained higher sequence or signer compromise stops publication for the affected domain. Recovery is explicit and auditable. The bot never merges main, accepts G7 or changes engine parameters to make a failed delivery succeed.

13. Security and operational readiness

Trust model

The engine authenticates the collector, not the external book's truth. A correct signature can contain dishonest prices/depth. Mitigations are approved rules, pinned identities, raw-evidence archives, independent reconstruction, restricted signing, alerting and external review. One venue is a deliberate single-source dependency; it is not decentralized price consensus. [R05; P]

Separate the observation signer from the gas-funded relay key. The feed identity must not also hold oracle/monitor/governance authority. Use a key backend proven to sign the exact raw digest; bind domains and policies; never log secrets. Back up journal/evidence and test restore before operating. Missing/compromised keys require the approved escalation plan, not an unimplemented rotation transaction.

Restrict outgoing hosts and request sizes; validate configured URLs to prevent SSRF; require TLS; bound provider responses, levels, memory and queue sizes. Review dependencies and lockfiles. Prevent untrusted external market descriptions or API payloads from controlling code, commands or signer configuration.

Health model

Expose per-market collection health, approval state, last authentic source time, newest signed/mined/finalized sequence, source age, headroom, invalid-depth reason, coverage/readiness and backlog. Read engine views at an identified block. Global health must not hide a failed worker.

Alert	Required operator response
Freshness/headroom exhausted	Stop fresh-price claims; investigate source or relay
Thin/wide/crossed book	Record invalid depth and notify risk/operator
Rules/mapping/status change	Quarantine market; request owner review
RPC/reorg/sequence mismatch	Reconcile canonical state before resuming
Writer/key/database failure	Stop signing; use tested recovery runbook
Required recorder stops before T	Notify settlement/oracle owners; preserve gap evidence

Capacity and cost [P]

For M active markets and cadence c seconds, observation rate is approximately M/c and daily calls M*86,400/c before invalid/retry overhead. At c=10 this is 8,640 observations per market per day, an illustration only. Measure RPC throughput, signer latency, nonce serialization, database I/O, rate limits and real gas before selecting M/c. Do not invent Monad fee estimates.

Use dry-run mode by default; transaction mode requires explicit approved environment/key/chain configuration. This document itself authorizes neither testnet nor production deployment, funding, contract activation or transaction sending.

14. Failure and lifecycle state machine

Bot states [P] - separate from engine enums

DISABLED -> VALIDATING -> SYNCING -> COLLECTING
-> INVALID_DEPTH
-> DEGRADED
-> QUARANTINED

Early engine halt -> RECORD_ONLY while INVALID history needs data
T / agreed recording completion -> STOPPED with retained evidence

COLLECTING means source data are trusted and a worker can build packets, not that the engine's mark is available. On recovery, obtain a complete approved snapshot and reconcile ordering before returning to collection. Engine window validity recovers under its own rules. Never create alternative economic Stage or PricingMode behavior. [R02, R07]

Condition	Bot action	Engine consequence / limit
Fresh complete valid book	Calculate, sign and relay with headroom	Valid sample; windows decide readiness
Authentic thin/wide/crossed book	Approved invalid summary + reason	depthValid=false; no coverage
Timeout or missing authentic time	Stop valid output; archive gap; alert	Previous carry expires within 30 s
Wrong token/rules/domain	Quarantine; do not sign	No admissible observation
Duplicate/backwards sequence	Reconcile journal/onchain state	Contract reverts
Future publishedAt	Hold safely or expire; fix clock	Contract reverts; zero tolerance
Source resolved/closed early	Record absence; approved missing-data path	Never manufacture post-close depth
Engine economically halted early	Continue valid index in record-only mode	No trading/funding resume
Signer/relay/DB failure	Stop signing; bounded recovery	No fake successful-delivery status

Separate economics from source observations

The bot does not decide that a football draw is INVALID, that an election is final or that an underlying crypto threshold was hit. Category rules inform admission/review; the authorized oracle supplies final outcomes. An external resolution event is an operational signal, not a settle() instruction.

For early INVALID, Eros retains [T-24h,T]. The feed should support independent recording through T; if the external source cannot remain usable, full coverage may fail. The explicitly listed new-market missing-index fallback becomes eligible at T+1h; the bot does not publish 0.5 to force it. Legacy unconditional-TWAP markets require separate product/governance resolution if unavailable. [R02, R03]

15. Independent arithmetic and adversarial vectors

Exact test oracle

Use independently derived Fraction arithmetic and fixed fixtures. Do not use the production calculator or Solidity output to generate expected values. Each test records method, units, rounding and config identity. The following vectors are illustrative test inputs, not production parameter selections. [R01; P]

Vector	Expected result
Price string 0.61	610000000000000000 WAD
Quantity 5 claims	5000 lots
Quantity 1.2349 claims	1234 reported lots if approved floor-total policy
Bid 0.59, ask 0.61; depth=N	Midpoint 600000000000000000
Bid 0.59, ask 0.640000000000000001; limit 0.05	Invalid: spread exceeds bound by 1 WAD atom
Same example ask=0.64; limit 0.05	Valid: equality passes; midpoint 0.615
Bid 0, ask 1, crossed or either side < N	Invalid under current PricingMath
Bid 0.590000000000000001; ask 0.590000000000000002	Midpoint 590000000000000001 WAD; half remainder floors

Impact-method discriminator [Q02]

N=10 claims. Bids: 6 at 0.60, 4 at 0.58. Asks: 6 at 0.62, 4 at 0.64. VWAP branch: bid=0.592, ask=0.628, mid=0.610, spread=0.036. Marginal branch: bid=0.58, ask=0.64, mid=0.61, spread=0.06. Midpoints happen to agree but validity can differ. This is why the impact definition must be frozen before claiming compatibility.

Window and clock vectors

Over a fully covered 300-second window, 100 seconds at 0.4 then 200 seconds at 0.7 yield 0.6, not the unweighted 0.55. Construct authentic samples no more than 30 seconds apart within each interval. A 31-second source gap contributes a one-second uncovered interval; the 300-second window is unavailable. Repeating an identical old timestamp adds no time coverage. Same-second replacement uses the later accepted sample with zero elapsed weight.

Boundary tests include uint64 limits, wrong fields/domain/signer, personal-message prefix, reordered ABI fields, malformed decimals, missing fields, duplicate levels, fractional size, zero size, future source/chain skew, source closure, negative-risk mappings and changed rules. Network chaos tests include 429, stale cache, missed stream hints, reconnect, clock skew, slow inclusion, crash at every outbox transition, two writers, nonce contention and reorgs.

16. Risk integration and verification levels

Levels must be reported separately

1. **Deterministic collector tests:** fixture data and independent math; no live source/engine claim. 2. **Polymarket live read-only soak:** authentic books, depth and source-time statistics; no transaction claim. 3. **Local real-ingress integration:** our signed packets enter the actual Pricelngress/ObservationStore/guards composition; CLOB/oracle status explicitly marked real or mocked. 4. **Approved live engine join:** deployed composition, canonical receipts and continuous valid windows; requires explicit environment/transaction authority. 5. **Full economic/production integration:** real book/oracle/factory/app, calibrated inputs, measured chain gas, external review and human release decision.

Existing risk regression commands [R]

These are future verification steps from contracts/, not runs performed to create this plan:

```
FOUNDRY_PROFILE=risk FORGE_SNAPSHOT_EMIT=false forge test \
  --match-path test/risk/B/B016.t.sol
FOUNDRY_PROFILE=risk FORGE_SNAPSHOT_EMIT=false forge test \
  --match-path test/risk/B/B017.t.sol
FOUNDRY_PROFILE=risk FORGE_SNAPSHOT_EMIT=false forge test \
  --match-path test/risk/B/B018.t.sol
```

B016 covers authentication/domain/time/order; B017 covers windows/gaps/replacement/INVALID retention; B018 covers bootstrap/readiness. These do not test our future bot by themselves. Add a reviewed CP-PRICE integration harness that uses submitObservation with independently signed packets. CombinedEngine.feed/feedStep/feedIndex are test stand-ins calling internal hooks; they cannot establish the live signed join. [R04-R07, R11]

Acceptance assertions

Require local digest == observationDigest; valid/invalid emitted flags and midpoint match expected values; wrong pins and domain revert; delayed time stays delayed; irregular TWAP equals independent reference; gaps block index readiness and funding does not restart across a stopped epoch; fresh recovery requires full windows. Verify callback leaves configuration immutable. After early halt, index recorder still works without resuming economic mutation; complete/gapped INVALID history follows its specified outcome.

Evidence contract

Record task/gate ID, git SHA, spec/ABI/source rules versions, exact command/cwd/exit code, nonempty test count, fixture hashes, seeds, source timestamps, block hashes, packet/digest/receipt IDs and real-versus-mock status. Missing tool/fixture/address means UNVERIFIED/BLOCKED, not PASS. Official risk gates write evidence and must be run by their owners on an approved worktree; human-only acceptance remains unchanged.

17. Calibration, rollout and production limits

Do not select N from a category label

Collect representative complete books for candidate markets in each category, including quiet intervals, active news/game periods and exception/closure periods. Sweep candidate N values using the approved impact method. Measure two-sided depth, spread, source-age distribution, gaps, rounding/precision, minimum size, changes near resolution and relay latency. Risk approves N/spread and the supported listing universe from these measurements. [R02, R05, R08]

N must respect source executable quantity constraints and Eros units. The same pinned depth rule also applies to the internal perp summary; include real CLOB measurements before economic approval. Do not increase maxSpread just to make stale/thin markets appear available or use the fixture's 500 lots/0.05 as a production default.

Staged rollout [P]

- Build deterministic parsing, exact arithmetic and signing vectors after PF-G0.
- Run read-only capture; unsupported mappings stay disabled.
- Integrate signed output into an approved local harness using real ingress.
- Select one approved representative market first; expand to sports/politics/crypto only as each mapping passes admission and source-soak criteria.
- Before production operation, approve runtime/dependencies, signer/relay ownership, source terms, rate limits, chain facts, gas/spend budget, alarms, recovery and audit.

A category can be supported by configuration even if no current live market in that category meets the listing gates. That is not a reason to bypass the gates or promise all Polymarket markets are usable.

Measured acceptance, not invented availability

Required technical condition: every claimed available engine index window has complete eligible coverage and correct integer price. Missing data must reliably expose unavailability. Production soak duration, load target, alert threshold, allowable unavailable periods and chain-finality assumptions remain named decisions [Q07, Q09, Q10]. A short sample or a green mock suite cannot certify 24-hour recorder availability.

Release separation

Initial Eros defaults remain all caps 1x and funding/recovery/conversion disabled. Feed completion does not enable leverage. Missing volatility/hazard/closeout/queue inputs, real counterpart joins, engine code-size facts, Monad gas, external audit and human release decisions remain separate production blockers. This plan includes handoff evidence for those owners; it neither supplies fabricated calibration nor authorizes launch. [R01, R02, R10]

18. Chronological milestones and dependency stops

Order / gate	Work and exit evidence	Must wait for
PF-G0	Scope owner, source contract, Q02-Q05/Q08 decisions, exact ABI and test vectors agreed	Human scope + risk/factory review
PF-G1	Config, parser and independent arithmetic tests pass; category dossiers supported	PF-G0; reviewed impact method
PF-G2	Full snapshots, time checks, shared adapter and read-only capture pass	PF-G1; approved source-time policy
PF-G3	Durable outbox, raw signer, multi-market sequencing and failure tests pass	PF-G2; signer/domain contract
PF-G4	Ordered local relay and real-ingress differential integration pass	PF-G3; approved local harness scope
PF-G5	Selected live-source read-only soak, calibration report and lifecycle tests reviewed	PF-G4; actual selected mappings
PF-G6	Operator/load/recovery/security review and deployment-ready manifest complete	PF-G5; production decisions
PF-G7	CP-PRICE handoff accepted with exact real/mock statuses; live join only if actually authorized/tested	PF-G6; authorized environment

What can proceed while counterpart code is unfinished

After CP-PRICE scope approval and decision freeze: exact reference fixtures, config/admission tooling, shared Polymarket reads, archives, dry-run packet construction, signer tests and local ingress tests. These do not require completed live CLOB/oracle matching/finality logic.

The real CLOB is required to prove final mark/bootstrap transitions and full trading/liquidation integration. The oracle is required for authentic halt/finality/lifecycle end-to-end acceptance. Factory/listing owners must provide actual immutable configuration and code/domain identities. Production operation requires authorized existing contracts, keys and transactions; no deployment is authorized by this document.

Task execution contract

PF001-PF028 use effort points as relative difficulty, not promised hours. CP-PRICE owns implementation; risk/oracle/factory/operator reviewers are named by role until humans assign people. Every task has dependencies, proposed allowed paths, output, future verification command, assertions and consuming gate. Proposed paths are relative to the approved CP-PRICE package root, NOT automatically permitted under this repository's CLAUDE.md.

Create the proposed test runner/scripts first; no PF command below exists or was run merely because it appears in the plan. Use test-first development, preserve peer work, log commands and failures, and cross-review behavior-changing decisions. Do not call scripts/check-task.sh PF001: the current risk runner accepts only A/B task IDs. Use a separate PF runner and evidence namespace after approval.

19. Backlog - agreement and foundations

PF001 - Scope and ownership freeze | 2 points

Owner: CP-PRICE lead + human; **depends:** none; **paths:** docs/ownership.md, docs/decisions.md only after permission. **Output:** named owner, permitted package/repository and review process; fixed versus open requirements list. **Verify:** proposed npm run check:scope. **Accept:** no forbidden risk-team fetch implementation, no deployment authority inferred, DEC rules intact. **Gate:** PF-G0. Stop code work until written scope is approved.

PF002 - Wire/signing contract freeze | 3 points

Owner: feed + risk reviewer; **depends:** PF001; **paths:** docs/wire-contract.md, fixtures/wire/, test/wire/. **Output:** eleven-field order/types, exact type string/domain, raw signature vectors and invalid-summary policy. **Verify:** proposed npm run test:wire. **Accept:** ABI matches source/export; no acceptedAt/input digest; wrong-prefix/domain vectors fail; I-3 explicitly confirmed. **Gate:** PF-G0.

PF003 - Market and pricing decisions | 4 points

Owner: feed + risk/oracle/factory; **depends:** PF001-PF002; **paths:** docs/decisions.md, docs/mapping-template.md, fixtures/rules/. **Output:** Q02-Q05/Q06/Q08 resolutions or explicit per-item blocks, representative category dossiers. **Verify:** proposed npm run check:decisions. **Accept:** no silent VWAP/marginal choice; timestamp, units, hashes and exception mapping are versioned; production values remain unapproved until evidence. **Gate:** PF-G0.

PF004 - Package/toolchain and evidence runner | 2 points

Owner: feed developer; **depends:** accepted PF-G0; **paths:** package.json, lockfile, tsconfig.json, scripts/check-pf.*, scripts/evidence.*, test/runner/. **Output:** pinned runtime/dependencies, strict bigint conventions, nonempty test discovery and task/gate evidence schema. **Verify:** proposed npm ci; npm run typecheck; npm run test:runner. **Accept:** exact versions recorded, no skipped/empty suite accepted, deterministic seeds/hashes and exit codes captured. **Gate:** PF-G1.

Block ordering: resolve public interface and scope first; do not build a signing service against an unagreed format. Agreement tests can be drafted as fixtures, but accepted economic/source behavior waits for reviewers. No actual owner names or approved values are fabricated in this plan.

20. Backlog - configuration and exact math

PF005 - Typed configuration and admission validator | 3 points

Owner: feed; **depends:** PF004; **paths:** src/config/, test/config/, schema/. **Output:** validated schema, required pins, category labels and approval identities. **Verify:** proposed npm run test:config. **Accept:** unsafe integers/placeholders, duplicate domain/source entries, missing approvals and listing mismatch fail closed; category does not select risk template. **Gate:** PF-G1.

PF006 - Category mapping dossiers | 3 points

Owner: feed + oracle/listing review; **depends:** PF003-PF005; **paths:** config/examples/, fixtures/rules/, docs/markets/. **Output:** disabled schematic sports/politics/crypto examples plus independently reviewed selected mappings when available. **Verify:** proposed npm run test:mapping. **Accept:** correct outcome token/label, deadlines and exception rules; negative-risk ambiguity rejected; short-horizon markets rejected; raw BTC/USD never substituted. **Gate:** PF-G1.

PF007 - Exact decimal and lot normalization | 3 points

Owner: feed; **depends:** PF004-PF005; **paths:** src/math/decimal.*, src/math/units.*, test/units/, reference/. **Output:** bounded bigint/rational parser, quote policy and conservative depth normalization. **Verify:** proposed npm run test:units; python3 -m unittest discover -s reference/tests. **Accept:** precision/overflow errors explicit; Fraction vectors independently derived; no floating-point financial operations; fractional aggregation cannot overstate available lots. **Gate:** PF-G1.

PF008 - Approved impact calculator | 4 points

Owner: feed + risk reviewer; **depends:** PF003, PF007; **paths:** src/math/impact.*, test/impact/, fixtures/impact/, reference/. **Output:** approved depth walk, rounding, midpoint and invalidity classification. **Verify:** proposed npm run test:impact plus independent reference suite. **Accept:** exact N/one-lot boundaries, spread equality/+1 atom, partial final level, duplicate levels, endpoints, min-size constraints and multi-level method discriminator. **Gate:** PF-G1.

Review stop: calculator arithmetic may be ready while production N remains uncalibrated. Do not turn fixture values into operational defaults. If risk selects a different impact method, version the method/rules fixtures and rerun affected tasks rather than mixing both methods under the same sourceRulesHash.

21. Backlog - source acquisition

PF009 - Metadata discovery and rule monitoring | 3 points

Owner: feed; **depends:** PF005-PF006; **paths:** src/adapters/polymarket/metadata.*, test/metadata/, fixtures/provider/. **Output:** pinned API schema, pagination, ID lookup and approved metadata/status cache. **Verify:** proposed npm run test:metadata. **Accept:** discovery never enables markets, array labels map correctly, changed rules/status quarantine, unknown/null fields fail according to policy, metadata dates do not override Eros T. **Gate:** PF-G2.

PF010 - Complete order-book adapter | 3 points

Owner: feed; **depends:** PF007-PF009; **paths:** src/adapters/polymarket/book.*, test/book/, fixtures/provider/. **Output:** bounded complete-book reads, identity verification and canonical levels. **Verify:** proposed npm run test:book. **Accept:** exact strings preserved, zero/duplicate/sorted levels tested, wrong token/condition rejected, partial/schema-invalid bodies never produce valid depth, hash archived. **Gate:** PF-G2.

PF011 - Source-time and freshness budget | 4 points

Owner: feed + risk reviewer; **depends:** PF010, approved Q03; **paths:** src/time/, test/time/, docs/timestamp-evidence.md. **Output:** evidence-based observedAt conversion, skew/headroom checks and repeat/delay policy. **Verify:** proposed npm run test:time. **Accept:** 29/30/31-second boundaries, future chain time, stale caches, unchanged timestamps and same-second behavior; no synthetic freshness; approved policy agrees with source semantics. **Gate:** PF-G2.

PF012 - Shared scheduler and stream hints | 3 points

Owner: feed; **depends:** PF009-PF011; **paths:** src/collect/, src/adapters/polymarket/stream.*, test/transport/. **Output:** market subscriptions, heartbeat, bounded periodic complete snapshots, global rate limiter/backoff and resync generation. **Verify:** proposed npm run test:transport. **Accept:** heartbeat is not price freshness, reconnect does not mix old generations, 429/timeouts bounded, one market cannot starve others, no unverified incremental book merging. **Gate:** PF-G2.

Safe first live step: read-only snapshots and diagnostics, with every market disabled for transaction output until the admission/signing/delivery gates pass. A missed stream hint is acceptable only if periodic complete snapshots and verified timestamp policy still meet coverage requirements.

22. Backlog - persistence and signing

PF013 - Evidence archive and transactional journal | 4 points

Owner: feed; **depends:** PF010-PF012; **paths:** src/storage/, migrations/, test/storage/. **Output:** records from page 13, evidence integrity hashes, sequence/outbox constraints and approved retention policy. **Verify:** proposed npm run test:storage. **Accept:** raw valid/invalid/gap data retained, transactions atomic, restores verified, database corruption/permission failures stop signing, no private keys in records. **Gate:** PF-G3.

PF014 - Observation builder and invalid summaries | 3 points

Owner: feed + risk reviewer; **depends:** PF002, PF008, PF011, PF013; **paths:** src/observations/, test/observations/. **Output:** exact input tuple, width checks, midpoint, approved invalid packet and publish-time semantics. **Verify:** proposed npm run test:observations. **Accept:** field order/units exact, fresh thin/wide summaries remain invalid, unknown source time never fabricated, dropped/coalesced samples preserve agreed invalid transitions. **Gate:** PF-G3.

PF015 - Raw digest and signer implementation | 4 points

Owner: feed + risk/security review; **depends:** PF014; **paths:** src/signing/, test/signing/, fixtures/wire/. **Output:** reviewed raw secp256k1 signer, recovery check and policy-bound key interface. **Verify:** proposed npm run test:signing. **Accept:** keccak/abi.encode exact, no EIP-191/712 prefix, low-s/v/serialization pinned, every domain/field mutation fails, signatures cannot be obtained for unapproved identities. **Gate:** PF-G3.

PF016 - Worker isolation and crash recovery | 4 points

Owner: feed; **depends:** PF013-PF015; **paths:** src/workers/, test/recovery/, test/concurrency/. **Output:** independent engine/source sequences, single-active-writer fencing, durable immutable packets and startup reconciliation. **Verify:** proposed npm run test:recovery. **Accept:** kill at every write/sign boundary, two-writer contention, restore lag, skipped sequences and uint64 exhaustion; never reuse a sequence for changed payload. **Gate:** PF-G3.

Gate evidence: compare generated packets to independent vectors before adding network sends. Production keys are not needed for deterministic signing tests; use explicitly test-only keys and never present those fixtures as approved operational signer identities.

23. Backlog - relay and real ingress

PF017 - Dry-run inspection CLI | 2 points

Owner: feed; **depends:** PF016; **paths:** src/cli/, test/cli/, docs/cli.md. **Output:** proposed commands validate-config, inspect-book, build-observation, verify-digest and read-only health. **Verify:** proposed npm run test:cli. **Accept:** dry-run default, no accidental send, fields/units/pins and reasons visible, secrets redacted, production placeholders cannot be enabled. **Gate:** PF-G4.

PF018 - Ordered transaction relay | 4 points

Owner: feed + operator; **depends:** PF016-PF017; **paths:** src/relay/, test/relay/, test/nonces/. **Output:** domain/preflight checks, shared nonce manager, simulation, bounded retries and spend/age budgets. **Verify:** proposed npm run test:relay. **Accept:** unknown send results reconciled, replacements keep packet unchanged, nonce ordering across markets tested, expired packet never retimestamped, RPC failures do not produce fake success. **Gate:** PF-G4. Sending remains disabled absent explicit authority.

PF019 - Receipts, logs and reorg reconciliation | 4 points

Owner: feed + production reviewer; **depends:** PF018; **paths:** src/reconcile/, test/reorg/, test/receipts/. **Output:** packet/log/digest matching, canonical block tracking, sourceState reconciliation and approved finality policy. **Verify:** proposed npm run test:receipts; npm run test:reorg. **Accept:** success versus depth validity distinguished, orphaned receipts rolled back locally, signed sequences never reused, unknown external higher sequence quarantines. **Gate:** PF-G4.

PF020 - Real-ingress differential harness | 5 points

Owner: integrator + risk reviewer; **depends:** PF019; **paths:** CP-PRICE test/integration/; risk-side harness paths only if separately assigned. **Output:** packets from bot/reference enter actual ingress/store/guards; independent expected events/windows and negative vectors. **Verify:** proposed npm run test:engine, plus existing B016/B017/B018 suites with the approved pinned toolchain. **Accept:** exact digest equality, valid/invalid cases, timing/sequence/domain failures, no CombinedEngine.feed bypass; peers identified real/mock. **Gate:** PF-G4.

Dependency stop: absence of an authorized real production engine is not a reason to fake an address. Local integration may pass while the live CP-PRICE join remains BLOCKED_BY_COUNTERPART. Only the risk owner changes/reruns risk-side tests and evidence under its current shared-turn rules.

24. Backlog - live source evidence and lifecycle

PF021 - Category live-source soak | 4 points

Owner: feed + risk; **depends:** PF020; **paths:** scripts/capture.*, artifacts/source-soak/, docs/soak-report.md. **Output:** approved read-only capture for selected sports/politics/crypto outcomes with exact source-age/depth/spread/gap statistics. **Verify:** proposed npm run soak:readonly -- --config APPROVED_CONFIG; npm run verify:soak. **Accept:** duration/targets explicitly agreed, no generated result, quiet and active periods included, unsuitable markets reported, no transactions. **Gate:** PF-G5.

PF022 - N/spread and cadence calibration report | 4 points

Owner: risk approves; feed supplies evidence; **depends:** PF021; **paths:** scripts/calibrate.*, artifacts/calibration/, docs/calibration.md. **Output:** candidate-depth comparison, executable-size/fee/quote assumptions and recommended operating budgets with evidence. **Verify:** proposed npm run verify:calibration. **Accept:** measured inputs, method/rounding fixed, both index and own-perp constraint acknowledged, 500-lot/0.05 fixtures not silently adopted. Production parameters remain blocked without signed owner review. **Gate:** PF-G5.

PF023 - Halt, closure and INVALID recorder tests | 4 points

Owner: feed + risk/oracle review; **depends:** PF020-PF022; **paths:** src/lifecycle/, test/lifecycle/, fixtures/lifecycle/. **Output:** record-only lifecycle, source-closure detection and requiredFeedUntil policy. **Verify:** proposed npm run test:lifecycle. **Accept:** continue independent samples after early halt through required T, no economic resume, gaps truthful, 24-hour complete/missing cases tested, fallback not forced by fabricated samples, no oracle callbacks sent. **Gate:** PF-G5.

PF024 - Load and fault-injection campaign | 4 points

Owner: feed + operator; **depends:** PF023; **paths:** test/chaos/, scripts/load.*, artifacts/load/. **Output:** multi-market throughput/headroom limits and failure evidence. **Verify:** proposed npm run test:chaos; npm run test:load. **Accept:** declared load target, global rate/nonce limits, bounded queues, slow signer/RPC, reorgs, clock skew, DB failure and source outages; zero unsafe fresh-price claims. **Gate:** PF-G6.

No arbitrary guarantee: this gate proves behavior under the recorded fixtures and measured sources; it cannot guarantee a venue never closes or markets remain liquid. Failed coverage leads to explicit unavailability and admission restrictions under the existing engine rules.

25. Backlog - operations and handoff

PF025 - Monitoring and runbooks | 3 points

Owner: feed + operator; **depends:** PF024; **paths:** src/observe/, docs/runbooks/, test/health/. **Output:** per-market metrics/alerts, dashboard/read-only status, restore/restart/expiry/closure/key-failure runbooks. **Verify:** proposed npm run test:health; npm run drill:recovery. **Accept:** unavailable != zero, engine views block-labeled, alert routing assigned, restore and duplicate writer drills succeed, recorder gaps visible. **Gate:** PF-G6.

PF026 - Security and release manifest | 4 points

Owner: feed + risk/security/production reviewers; **depends:** PF022-PF025; **paths:** docs/security.md, docs/release-manifest.json, artifacts/review/. **Output:** runtime/lock/code hashes, source rules/mappings, signer/relay identities, chain/finality/gas budget, data terms review, retained limitations and approvals. **Verify:** proposed npm run check:release. **Accept:** no missing approved values, signing backend supports raw digest, no hidden key rotation, least privilege, external review path; no launch authorization inferred. **Gate:** PF-G6.

PF027 - Counterpart handoff and authorized live join | 5 points

Owner: integrator + feed/risk/factory; **depends:** PF026 plus explicit environment/transaction authority; **paths:** artifacts/handoff/, docs/handoff.md; counterpart evidence through owners. **Output:** actual deployment/ABI/pins and signed observation receipts, readiness and gap recovery evidence if an authorized engine exists. **Verify:** proposed npm run verify:handoff; approved live smoke command only after authority. **Accept:** real signed path, exact log/digest/pins, live source and real engine distinguished from mocks; real CLOB/oracle needed for full economics. **Gate:** PF-G7. Without authority/address, live part is BLOCKED, not waived.

PF028 - Final review and reproducible delivery | 3 points

Owner: independent reviewer + CP-PRICE lead; **depends:** PF027; **paths:** docs/final-review.md, artifacts/final/, README.md. **Output:** accepted packet format, task evidence index, operator instructions, category support matrix and limitations. **Verify:** proposed npm run verify:all; clean-environment replay. **Accept:** all nonempty tests/evidence reproduced, unresolved blockers disclosed, production/release status separate; no self-approval fingerprints or human-only gate changes. **Gate:** PF-G7.

At the end: a bot that provides correct, authenticated event-price observations for explicitly approved markets and truthfully fails closed when it cannot. Not a promise to support every event on Polymarket, nor permission to ignore Eros listing/rule constraints.

26. Decisions and exact questions for owners

ID / owner	Question to resolve	Blocks
Q01 / human	Who owns CP-PRICE; where may code/tests live; which branch/review process is authorized?	Implementation
Q02 / risk + feed	Is impact price quantity-weighted execution price or marginal N price? Approve bid/ask rounding, fee treatment and depth definition.	Calculator/hash
Q03 / risk + feed	What does vendor timestamp mean; how do we prove observedAt freshness on unchanged/cached books; what is publishedAt policy?	Freshness/signing
Q04 / risk + factory	What canonical source rules/mapping document is hashed, and how are sourceHash/rulesHash/indexRulesHash related?	Config/init
Q05 / risk + feed	Approve pUSD/USDC quote normalization, source size/min-order precision and lot-conversion boundary.	Numeric contract
Q06 / oracle + listing	Which exact initial event/token mapping is approved; how do cancellation/50-50, runoff and rules clarifications compare?	Each market
Q07 / risk + operator	Approve measured cadence, signing/inclusion headroom, metadata-age bound and stale/delayed delivery policy.	Live reliability
Q08 / risk + feed	Approve invalid-summary fields and delivery priority; how do untrusted-time failures invalidate safely without fabricated time?	Failure correctness
Q09 / production	Provide chain/RPC/code/ABI identities, raw signer backend, relay budget and canonical finality/reorg policy.	Live join
Q10 / risk + production	Approve soak duration/load/availability targets, calibrated N/spread, retention, monitoring owner and release criteria.	Production

Copy-ready risk-team request

Please confirm I-3's exact raw digest and eleven-field Observation ABI; choose and document impact-price method/rounding/depth semantics; agree observedAt/publishedAt policy with actual Polymarket timestamp evidence; approve invalid summaries and delayed/coalesced delivery behavior; and identify the canonical indexRulesHash document. Please provide or review an approved listing with marketId, chain/engine/code/ABI, indexSourceId/indexSigner/indexRulesHash, N/maxSpread, T and INVALID policy. Production N/spread will be proposed only after captured-depth calibration.

Other handoffs

Oracle/listing: approve exact question/outcome/exception mapping and ongoing source closure policy; do not ask the bot to resolve events. Factory: wire the approved immutable source/domain/config; no placeholder production addresses. Production: decide measured operating profile and authorized transaction environment. Internal risk teammate: B-D02/A-I01 reviews, A043 refresh and risk G7 remain their separate task. No dependency is marked accepted on their behalf.

All open decisions become versioned records with owner, date, evidence and downstream reruns. An unanswered semantic question stops that item; it does not invite the developer to pick silently.

27. Definition of done and first implementation slice

Bot-complete acceptance checklist

- CP-PRICE scope and exact source/output contract are approved.
- One shared service handles configured sports/politics/crypto workers with isolated state; exact event mappings and supported/blocked category cases are recorded.
- Source identity/rules/time/unit validation cannot be bypassed by category labels or config edits.
- Independent fixtures prove exact impact math, conservative normalization and engine-valid midpoint.
- Raw digest equals the actual engine digest; pinned signer recovers correctly; wrong domains/fields/prefixes fail.
- Sequence allocation, signing persistence, writer fencing, nonces, retries and reorg recovery survive fault injection.
- Signed packets use submitObservation; matching receipts/events and real/mock status are recorded. No internal feed() shortcut substitutes for integration.
- Genuine fresh valid coverage produces the expected index; invalid/stale/source failures produce explicit gaps/unavailability. Old data are never refreshed by receipt time.
- Early halt/closure/INVALID recording follows the immutable listing and does not resume economics or invent fallback prices.
- Monitored operating profile, approved N/spread, keys, retention, terms and runbooks are delivered; actual production authority remains separately recorded.

Supported does not mean always available

Correct behavior includes refusing a bad mapping and stopping fresh output during outages. Do not define success as forcing a price to exist under all conditions. A source with frequent stale/depth gaps can be technically readable yet unsuitable for the desired operational availability; admission/calibration must make that visible.

First chronological slice after approval

1. PF001-PF003: record ownership and resolve wire/math/time/hash/units questions; do not choose production N yet. 2. PF004-PF008: build package runner, disabled category config examples and independently tested exact calculator. 3. PF009-PF012: implement complete-book read-only collection and source-time evidence. 4. PF013-PF020: add durable builder, raw signer, recovery, CLI and real-ingress local tests. 5. PF021-PF028: collect source/calibration evidence, harden, obtain reviewed production inputs and perform only explicitly authorized live acceptance.

Current artifact status: plan complete for review; all bot tasks NOT STARTED; integration decisions OPEN; no production-ready bot or live acceptance claimed. Existing risk G7, merge, contract-size/Monad gas, real CLOB/oracle/factory/app and external-audit/release dependencies remain outside this artifact's completion claim.

28. Source register

Repository sources - baseline SHA on cover

ID	Path and role
R01	CLAUDE.md - scope, shared workflow, honesty, math and no deployment
R02	docs/spec/risk_spec.md - sections 1-5, 8/INVALID, 10, CP-PRICE and release gates
R03	docs/counterpart-contracts.md; docs/spec/oracle_interface.md - counterpart/INVALID boundaries
R04	contracts/src/interfaces/IPriceSource.sol - exact input ABI/event/units
R05	contracts/src/pricing/PriceIngress.sol; ObservationStore.sol - raw digest, source pins, ordering/store
R06	contracts/src/math/PricingMath.sol - validity, midpoint and time integrals
R07	contracts/src/pricing/RiskPricing.sol; SourceGuards.sol; risk/MonitorPolicy.sol - readiness/guards
R08	contracts/src/interfaces/IMarketConfig.sol; risk/RiskContextPort.sol; math/LifecycleMath.sol - config/horizons
R09	docs/merge/STATUS.md; docs/merge/B-assumptions.md I-3/I-4; docs/spec/implementation_plan.md
R10	docs/risk/HANDOFF.md; artifacts/risk/release-manifest.json; docs/spec/integration_gates.json
R11	contracts/test/risk/B/B016.t.sol, B017.t.sol, B018.t.sol, B019.t.sol; contracts/test/integration/CombinedEngine.sol

Official Polymarket sources - checked 2 October 2026

W01 - Prices and Order Books: <https://docs.polymarket.com/market-data/prices-order-books>

W02 - Real-Time Data: <https://docs.polymarket.com/market-data/realtime-data>

W03 - Discover Markets: <https://docs.polymarket.com/market-data/discover-markets>

W04 - Market Details: <https://docs.polymarket.com/market-data/market-details>

W05 - Resolution: <https://docs.polymarket.com/concepts/resolution>

W06 - Polymarket USD: <https://docs.polymarket.com/concepts/pusd>

These links document the provider, not Eros production approval. Record and hash the exact schemas/docs used when implementing, since APIs can change. Official documentation index: <https://docs.polymarket.com/llms.txt>. Proposed engineering policies in this plan are not attributed as provider guarantees.

29. Self-review, limitations and artifact evidence

Requirements cross-check

Check	Plan coverage / conclusion
Exact eleven input fields and ordering	Page 11; compared to IPriceSource.sol
Raw digest and chain/engine domain	Page 12; compared to PriceIngress.sol
No acceptedAt/depthValid/digest input	Page 11; event-output distinction explicit
N/spread/interior-price/midpoint checks	Page 10; PricingMath equality/endpoint rules
30/300/60/900/86400-second responsibilities	Pages 3, 9, 16, 18; risk owns aggregation
Immutable source/signer/N/spread and listing horizon	Pages 5-6, 12; config/code constraints
Same N/spread for index and internal perp	Pages 10, 19; joint calibration required
No unsupported impact/time assumptions	Q02/Q03 explicitly block dependent acceptance
True signed integration, not feed()	Page 18 and PF020/PF027
Category configurability without three bots	Pages 4, 6-7 and PF006/PF016
No fake live/pass/calibration/deployment claims	Every gate; current status below

Prior read-only observations [O] - not a new soak

Earlier in this conversation, public Gamma queries returned sports/politics metadata and complete CLOB reads succeeded. At approximately 13:08 UTC on 2 October 2026, the T1-versus-JD Gaming outcome snapshot had best bid/ask 0.61/0.62 and source age about 0.26 seconds. The Lula-election YES snapshot had 0.44/0.45 and age about 30.45 seconds; a later read had age about 14.86 seconds. Both reported minimum order size five claims. These point observations motivate timestamp/min-size review; they do not certify N, streaming continuity, full windows, semantics or production availability.

What was verified for this artifact

Repository/interface reads and remote fetch completed; no source code or shared status/approval record was edited. Exact ABI/type-string comparison, independent illustrative arithmetic checks, PDF text/layout checks and rendered-page inspection are performed as artifact QA. Bot/risk regression suites, live stream/soak, deployment and signed live transactions were not performed and are not claimed.

Only the final PDF is a requested new repository artifact. Rendering/build intermediates stay in temporary storage and are cleaned after inspection. Existing contracts/foundry.lock remains untouched. This plan is ready for owner review; the concrete next action is PF001-PF003, not an unapproved implementation or production send.